GraSTI-ACL Reproduction — Verified Solution Document

Repository: **Novelex/fMRI-DP** @ **a159a17** | Paper: He et al., *Medical Image Analysis* 107 (2026) 103815

Every item below was checked against the repository source and the paper text. Verdicts: **CONFIRMED**, **CORRECTED**, or **DO NOT DO THIS**.

Summary of verification

#	Claim in solution.md	Verdict
1	Rank-3 embedding is "the same as the paper (Eq. 17/18)"	**CORRECTED** — paper has an activation; your code removes it
2	Concatenate 90 PCC + 3 ALFF = d 93	**CONFIRMED** as Arm B — but it is a stated deviation, try Item 1 first
3	Need beta_convention=literal + gamma_orig_mode=signal_strength	**CONFIRMED** — that pair is unreachable with the current 3-way flag
4a	M_ij should be raw ALFF, not encoder embeddings	**CONFIRMED** — good catch, Eq. 4 with d = 3
4b	Add view_optimizer.step() before zero_grad()	**DO NOT DO THIS** — it inverts the adversarial objective
5	Set dyn_weight = None to skip the unused ToyNet	**CORRECTED** — right idea, but as written it crashes
6	Two betas in code, one in paper	**CORRECTED** — the paper has two, the code has three

Item 1 — Rank-3 embedding | CORRECTED

Your text: "Yes it is correct and same is the thing done in paper we can see Eq 17 and 18."

> **This is the one item that must change before submission.** The rank-3 collapse is a *deviation* from the paper, not a faithful implementation of it.

What the paper actually specifies

Eq. 17 reads:

```
X_Topo = GCN(X, A)
GCN(X, A) = σ( D■^(-1/2) A■ D■^(-1/2) X Θ )
```

and the text immediately after states: *"Θ is the parameter of network, and σ is the activation function, such as ReLU."* **The activation is part of Eq. 17.**

What your code does

```
# TA_encoder.py, GCN loop
if i == self.num_gc_layers - 1:
    # remove relu for the last layer
    x = F.dropout(x, self.drop_ratio, training=self.training)
else:
    x = F.dropout(F.relu(x), self.drop_ratio, training=self.training)
```

Every launch script sets `--num_gc_layers 1` (run_training_emb*.sh:26, gamma_mode_array.sh:34, reg_lambda_sweep_array.sh:33). With one layer, layer 0 *is* the last layer, so the ReLU is skipped and there is **no activation anywhere**. Your own argparse default (line 518) is already 2 — the shell scripts override it.

Why that collapses the representation

With no nonlinearity, BatchNorm affine at eval, sum pooling, and λ = 1e-4 killing the attention branch, the whole path is:

```
z = 1■ · A■ · X · Θ + 90b        where X is 90×3
```

1■A■X is a vector of exactly three numbers, and everything after it is affine. Simulated on 400 subjects through this exact path:

```
singular values: 2.569e+02  2.407e+02  2.217e+02  1.224e-12  1.397e-13 ...
variance in first 3 PCs      : 100.0000 %
effective rank (>1e-6 × s1) : 3 of 128
```

Twelve orders of magnitude between the 3rd and 4th singular value. This is the mechanism behind AUC 0.513 / 0.512 / 0.506 at emb_dim 32 / 128 / 256 — a 3-wide pipe does not widen when you enlarge the tank.

Fix

--num_gc_layers 2. Two layers puts Eq. 17's σ back on layer 0 and breaks the collapse *with no deviation from the paper*. Do not use 3 or more: on a complete 90-node graph one hop already reaches every node, so extra layers only oversmooth.

> **Correct wording for the document:** "d = 3 node features are faithful to Section 4.1. The rank-3 collapse is caused by `num_gc_layers=1` combined with the removed last-layer ReLU, which omits the activation σ that Eq. 17 specifies. Setting `num_gc_layers=2` restores it."

Item 2 — d = 93 node features | CONFIRMED as Arm B

Concatenating each subject's 90-dim FC row with the 3 ALFF bands is a sound idea: it makes FC enter as *information* rather than only as a smoothing kernel. But Section 4.1 states `d = 3` explicitly, so this is a **stated deviation** and must be labelled as one.

> **Run Item 1 first.** The collapse is caused by the missing activation, not by d = 3. The paper reaches 62.56% on MDD with d = 3, so d = 3 is not inherently the bottleneck. Only if `num_gc_layers=2` leaves you at ~52% does Arm B become necessary.

Item 3 — Beta convention and gamma mode | CONFIRMED

You are right on both counts: the two settings exist, and the combination you want is **not reachable**. `--gamma_mode` is a 3-way switch, not a 2×2 grid:

```
beta_convention  = "literal" if args.gamma_mode == "literal_beta" else "reversed"
gamma_orig_mode  = "signal_strength" if args.gamma_mode == "signal_strength" else "one"
```

--gamma_mode	beta_convention	gamma_orig	λ at eval	Status
baseline	reversed	1.0	1e-4	attention DEAD
literal_beta	literal	1.0	1.0	full attention
signal_strength	reversed	mean(|W|) ~0.39	0.61	alive
(what you want)	**literal**	**mean(|W|)**	**0.39**	**unreachable**

Fix: replace the single `--gamma_mode` with two independent flags, `--beta_convention {literal,reversed}` and `--gamma_orig_mode {one,signal_strength}`. All four combinations then become testable.

> **Disclose this in your methods.** Eq. 18 writes $\lambda \sim B(\gamma, 1-\gamma)$, giving $E[\lambda] = \gamma$. Section 3.3's prose says the opposite: *"as the connectivity of the graph diminishes... focus more on global information"*, which requires $E[\lambda] = 1-\gamma$. The paper contradicts itself. Choosing `literal` follows the equation over the prose — a defensible choice that must be stated, and ideally ablated both ways.

Second point worth knowing: $B(\gamma, 1-\gamma)$ has both shape parameters below 1 for *every* γ in (0,1), so it is always U-shaped. The paper says it chose $x=\gamma$, $y=1-\gamma$ specifically to avoid Mixup's 0.4/0.4 which *"bias the weight towards two extremes"* — but its own choice is equally extreme. Your deterministic-λ-at-eval fix already neutralises this at inference; it still applies during training.

Item 4a — M_ij from raw ALFF | CONFIRMED

Correct, and a good catch. Eq. 4 defines `M_ij = σ(v_i · v_j)` as *"the dot product of the ith and jth nodal feature vectors v_i and v_j in R^d"*, and Section 4.1 fixes `d = 3`. So `v_i` is the **raw 3-band ALFF vector**, not an encoder output.

Your code instead uses 32-dim encoder embeddings:

```
# view_learner.py
emb_src = node_emb[src]; emb_dst = node_emb[dst]
edge_prod = torch.sum(emb_src * emb_dst, dim=1)
```

Fix: compute `M_ij` from `batch.x` directly. It becomes a constant per subject, which is fine — see Item 4b.

> **One inconsistency to disclose:** if M is constant, Eq. 15's `max_Θ I_N` has nothing to optimise, since I_N then depends only on Φ. That is the paper's inconsistency, not yours. One sentence in methods covers it.

Item 4b — "add view_optimizer.step() before wiping it" | DO NOT DO THIS

> **This would break the adversarial training.**

The reasoning: `model_loss = calc_loss(x, x_aug) + ce_lambda * ce_loss`. And `x_aug` depends on `batch_aug_edge_weight`, which depends on `edge_logits_vl`, which comes from the view learner. So `calc_loss` **also** has a gradient path into the view learner.

Calling `view_optimizer.step()` after `model_loss.backward()` would therefore make the view learner **minimise** the contrastive loss. Its job under Eq. 15 and Eq. 21 is to **maximise** it. The augmenter would start collaborating with the encoder instead of opposing it, and the min-max collapses.

Correct fix. With M_ij taken from raw ALFF (Item 4a), L_CE has a gradient path only to Φ. Eq. 15's `min_Φ` then says Φ should minimise it, so move it into `view_loss` with a minus sign — `view_loss` is maximised, so a minus

sign means minimise L_CE:

```
view_loss  = model.calc_loss(x, x_aug) \
           - args.ce_lambda  * ce_loss \
           - args.reg_lambda * reg \
           - args.kld_lambda * kld_loss

model_loss = model.calc_loss(x, x_aug)      # ce_loss removed
```

This matches the paper's own description: *"maximizing the first term makes the optimized adjacency matrix more closely resemble the nodal feature vector dot product"* — maximising that mutual information means minimising the cross-entropy. No extra optimiser step, no inverted objective.

Item 5 — dyn_weight = None | CORRECTED

The diagnosis is right: `model` owns a 415k-parameter ToyNet whose output is discarded at all four call sites, yet `GInfoMinMax.forward` runs `get_mu_std_logits` — the batch×90 Python loop — every time. Four wasted passes per batch.

But the fix as written will crash. The None branch returns a 2-tuple:

```
# ginfominmax.py
if dyn_weight is None:
    return z, node_emb        # <-- TWO values
```

while the call sites unpack three:

```
x, _, edge_logits = model(...)   # line 216
x_aug, _, _       = model(...)   # line 252
x, _, _           = model(...)   # line 304
x_aug, _, _       = model(...)   # line 337
```

Passing `dyn_weight=None` gives `ValueError: not enough values to unpack (expected 3, got 2)`.

Fix: pass `dyn_weight=None` *and* change those four lines to unpack two values. Line 216's `edge_logits` is captured and never used, so nothing is lost. Expect roughly a 3× speedup. Verify accuracy is bit-identical afterwards — if it moves, something else was reading those logits.

Item 6 — How many betas | CORRECTED

Not "two in code, one in paper". The paper has **two** distinct things named beta, and the code has **three**:

Symbol	Where	Role	Status in your code
β	Eq. 2, 4, 13, 14	IB hyperparameter on the KL / MI term	implemented as --kld_lambda
B(·,·)	Eq. 18	Beta distribution generating λ	implemented as beta_convention
beta = 0.5	GraSTIACL.py, TA_encoder.py	none — not in the paper	dead: threaded through, never read

I grepped the whole of `TAEncoder.forward`: `beta` appears only in comments and in the unrelated `self.beta_convention`. The epoch-end `beta = np.random.beta(...)` updates nothing.

Fix: delete the dead `beta` threading. In methods, state that the paper's β (Eq. 4/13/14) is implemented as `--kld_lambda`, consistent with Appendix A.1's λ_KL in [0.001, 0.002, 0.003]. Do not claim β is implemented as a separate parameter.

Execution order

#	Change	Effort	Item
1	--num_gc_layers 2 in all launch scripts	1 character	1
2	Split --gamma_mode into two independent flags	~6 lines	3
3	M_ij from raw batch.x; move L_CE into view_loss with a minus sign	~8 lines	4a, 4b
4	dyn_weight=None on the four calls + unpack two values	~6 lines	5
5	Delete the dead beta threading	~5 lines	6

Four checks before spending GPU hours

Check	Pass condition	A failure means
SVD of the embedding matrix	more than 10 components above 1e-6 × s1	Item 1 not fixed
λ at eval	greater than 0.1	Item 3 not fixed
calc_loss alone (not model_loss)	below log(batch – 1), and falling	encoder cannot match a subject to its own view
trained test acc vs "Before training"	strictly above it	training is doing nothing — kill the job

log(batch – 1) = **3.43 at batch 32**, **4.84 at batch 128**. Do not compare calc_loss across batch sizes — the chance baseline moves.

Add these two assertions so none of this can fail silently again:

```
assert (sv > 1e-6 * sv[0]).sum() > 10, f"embedding collapsed: rank={(sv>1e-6*sv[0]).sum()}"
assert lambda_ > 1e-3, f"attention dead: lambda={lambda_:.2e}, gamma={float(gamma):.4f}"
```

Context for your expectations

Table 3 of the paper reports on MDD (multi-site, psychiatric, n = 486 — the closest analogue to your 956 ABIDE subjects):

Method (paper, MDD)	Accuracy
GAT	48.75%
GCN	**52.05%**
AD-GCL	58.22%
A-GCL	59.65%
GraSTI-ACL	**62.56%**

Your GNN currently sits at **51.5%** — the paper's GCN row, almost exactly. That is the expected outcome when the mechanisms separating GraSTI-ACL from a plain GCN are inert: with λ ~ 0 the TAE reduces to a GCN, and with L_CE carrying no gradient the node information bottleneck is not optimised. After the five fixes, the target is roughly **62%**.

> It may still sit below your raw-FC elasticnet at 66.5%. **That is a legitimate finding, not a failure** — Section 5.1 of the paper concedes the same point about MLP baselines on region-averaged features. A GNN compressing 90×3 ALFF through a graph into 128 dimensions is not obliged to beat an SVM reading 4005 numbers directly.

Two results are worth reporting either way: (1) the reproduction itself, with the rank-3 SVD diagnostic as a concrete, verifiable failure mode of the released architecture; and (2) the honest comparison of classical ML at 66.5% against the GNN on identical features and identical folds.